

TEMA 1: INTRODUCCIÓN A LOS SISTEMAS EN TIEMPO REAL

1. DEFINICIONES

1.1 DEFINICIÓN DE “TIEMPO REAL”

Sistema que debe producir unas salidas como respuestas a unas entradas dentro de unos límites de tiempo específicos. Es decir, ni antes ni después de estos límites. No necesitamos el mismo tiempo de respuesta en unas aplicaciones que en otras y dentro de una misma aplicación habrá tareas para las que el tiempo de respuesta es crucial (se puede quemar la CPU) mientras que en otras es simplemente recomendable (que un juego vaya a tirones).

1.2 DIFERENCIAS ENTRE UN SISTEMA EN TIEMPO REAL Y UNO GENERAL

Los STR son sistemas en los que la precisión del sistema depende no solo del resultado lógico sino del tiempo en el que el resultado se produjo. Una respuesta óptima tardía es una respuesta errónea (o inválida). Una respuesta buena a tiempo es una respuesta aceptable. Da igual que sea justo cuando empieza el tiempo límite o que sea en el último momento.

Los Sistemas Informáticos generales tienen como finalidad obtener la mejor respuesta correcta sin tener en cuenta el tiempo que tardan en obtenerla... En el momento que se le pone algún límite temporal, ya sería de Tiempo Real.

1.3 TIEMPOS LÍMITE O DEADLINE

Los tiempos límite o “Deadline” son los límites temporales que tienen las tareas para responder. En algunos casos esos “Deadlines” son fijos mientras que en otros pueden variar en función del entorno físico y lógico. Hay “Deadline” asociados al tiempo absoluto mientras que hay otros asociados al relativo (con respecto a otra tarea, con respecto a un determinado evento, etc.)

2. ESTRUCTURA DE UN SISTEMA EN TIEMPO REAL

2.1 SENSORES Y ACTUADORES

Los sensores miden el “estado” del proceso bajo control y su entorno (presión, temperatura, velocidad, humedad, etc.) Los actuadores provocan cambios en el proceso bajo control o en el entorno, modificando elementos (motores, luces, etc.). La velocidad de muestreo determina la precisión de las mediciones. La latencia de los actuadores, junto con el límite máximo de tiempo de funcionamiento, determina otros aspectos de los sistemas.

2.2 SISTEMA EJECUTIVO

Este sistema toma los datos de los sensores, los procesa y produce respuestas sobre los actuadores que a su vez modifican el estado y cambian los valores medidos por los sensores. Esto se puede realizar con un controlador PID. Cuando hay más de una tarea a realizar se utiliza un sistema computacional.

Debe existir un balance entre la velocidad lenta de los sensores y actuadores y una respuesta suficientemente rápida para realizar correctamente los algoritmos de control.

2.3 SISTEMA DISPARADOR DE EVENTOS

Este sistema es uno de los que los diferencia de los sistemas estándar. Se encarga de generar secuencias de “disparos” (triggers) que arrancan la computación de determinadas tareas. Debe ser muy precisos y suelen tener asociado un reloj de tiempo real y/o un “watchdog timer” (para evitar bloqueos del sistema). Se trata de un disparador de eventos asociado al tiempo.

3. TIPOS DE TAREAS

3.1 TAREAS PERIÓDICAS VS TAREAS APERIÓDICAS O EVENTUALES

En general, la mayoría de las tareas presentes en un STR son tareas que se repiten periódicamente. Suelen estar acotadas en su tiempo de computación, por lo que el diseñador conoce cuánto tiempo van a tardar en ejecución y se pueden planificar a priori.

Aquellas tareas que no presentan un periodo constante se denominan aperiódicas. Un caso especial son las que están asociadas a determinados eventos generados muy esporádicamente, a éstas se les denomina tareas eventuales o esporádicas.

3.2 TAREAS CRÍTICAS VS TAREAS NO CRÍTICAS

Las tareas se clasifican en críticas y no críticas en función de las consecuencias que tenga la no obtención de un resultado antes de su “deadline”. Las tareas críticas son aquellas en las que en caso de sobrepasarse su “deadline” puede ocurrir una catástrofe.

Las tareas no críticas son aquellas en las que en caso de sobrepasarse su “deadline” solo provoca una pérdida de datos o una degradación en la calidad del sistema.

4. CLASIFICACIÓN DE LOS SISTEMAS EN TIEMPO REAL

4.1 TIEMPO REAL ESTRICTO (HARD REAL-TIME)

Son aquellos sistemas en las que es obligatorio que las tareas obtengan su respuesta antes de su “deadline”. Ejemplos: Aviones, centrales nucleares, control de fuegos de un edificio...

4.2 TIEMPO REAL FLEXIBLE (SOFT REAL-TIME)

Son aquellos en los que es admisible que alguna tarea supere su “deadline” y el sistema continuará funcionando. Ejemplos: Sistemas de adquisición de datos, tarjetas de Sonido...

5. APLICACIONES DE LOS SISTEMAS EN TIEMPO REAL

5.1 SISTEMAS DE CONTROL

- Control de tráfico vial/aéreo.
- Control de seguridad en centrales nucleares.
- Control de procesos.

5.2 SISTEMAS BIOINSPIRADOS

- Sistemas basados en redes neuronales, o algoritmos genéticos o que simulan sistemas biológicos o naturales.

5.3 SISTEMAS DE ADQUISICIÓN O PROCESAMIENTO ESPECÍFICO O AVANZADO DE DATOS

- Tarjetas gráficas, tarjetas de sonido, etc.

5.4 OTROS SISTEMAS EN TIEMPO REAL

- Muy importantes y de gran trascendencia en los últimos tiempos, los sistemas empujados.

TEMA 2: LENGUAJES PARA APLICACIONES EN TIEMPO REAL

1. CARACTERÍSTICAS DESEABLES EN LOS LENGUAJES PARA APLICACIONES EN TIEMPO REAL

Tiempo real se refiere a que lo que garantiza es el tiempo. Se puede usar cualquier tipo de lenguaje de programación, pero esto no significa que todos sean buenas opciones. Determinados lenguajes facilitan la programación de aplicaciones de Tiempo Real:

- Hay lenguajes específicamente diseñados para dar soporte concurrente a las tareas: ADA, Occam, Java-RT
- Hay lenguajes que consiguen la concurrencia mediante extensiones del lenguaje: C/C++

1.1 SEGURIDAD

Efectividad del compilador y del sistema en ejecución para detectar automáticamente errores de programación. Cuando se crea una aplicación en tiempo real, se debe garantizar que da respuesta y que si no la da es porque esté estimado que puede no darla. Características que lo facilitan:

- Datos tipificados (no sirve *Python*).
- Declaración previa de todas las variables ya que es preferible perder memoria a perder tiempo y que no se cumpla el *deadline*.
- Sintaxis no ambigua.

1.2 LEGIBILIDAD

Facilidad para entender el programa sin necesidad de documentación externa adicional. Habitualmente se consigue:

- Uso de comentarios descriptivos, aunque su uso no garantiza que sea legible como por ejemplo si estos comentarios están en japonés.
- Pocas restricciones en la nomenclatura de las variables y funciones.

1.3 FLEXIBILIDAD

Capacidad de un lenguaje para describir todo tipo de funcionalidad sin necesidad de recurrir a otros lenguajes o extensiones del lenguaje (habitualmente, a preprocesadores, librerías o al ensamblador). C, por ejemplo, no es flexible ya que necesita extensiones.

Especialmente:

- Acceso al hardware.
- Control de procesos y tareas.
- Control de recursos.

1.4 PORTABILIDAD

Mide la capacidad que tiene un lenguaje para ejecutar el mismo código fuente en diferentes máquinas. No es tan importante como para aplicaciones “estándar”, ya que se asume que existen restricciones con respecto al hardware en el que se ejecutan las aplicaciones de TR. En particular:

- Problemas con tamaño de registros.
- Problemas con la precisión en la representación numérica.

1.5 SIMPLICIDAD

Mide la capacidad de un lenguaje para describir con pocas sentencias de baja complejidad las diferentes funciones que se desean que ejecute una aplicación. La simplicidad reduce la probabilidad de errores, reduce el costo y aumenta la seguridad y la portabilidad. Un ejemplo de simplicidad sería SQL y uno que no es simple sería ensamblador.

1.6 MODULARIDAD

Mide la capacidad de un lenguaje para definir subrutinas dentro de su código de manera que estas subrutinas se puedan desarrollar como módulos independientes capaces de interactuar entre sí. Imprescindible:

- Control estricto de los parámetros de entrada y salida.
- Ocultación de la información de un módulo a los demás.
- Especificación clara y precisa tanto de la función que realizan como de la interfaz.
- Posibilidad de compilación y depuración separada de cada módulo (no tan importante).

1.7 EFICACIA

Mide la capacidad de los lenguajes para garantizar una ejecución rápida con un rendimiento garantizado que sea capaz de cumplir con las restricciones temporales. Siendo más importante el rendimiento garantizado que la ejecución rápida. No es importante la rapidez si se mantiene el tiempo. Énfasis en:

- Tamaño del código.
- Velocidad de ejecución. A más velocidad menos espacio y viceversa.
- Uniformidad en los tiempos de ejecución.
- Deben permitir la medición exacta de los tiempos de ejecución de cada tarea.

Dificultad en la predicción del tiempo de ejecución:

- Variables que varían dinámicamente su tamaño (cadenas, etc.).
- Bucles tipo while.

2. ADA COMO LENGUAJE PARA TIEMPO REAL

2.1 INTRODUCCIÓN A ADA

ADA es un lenguaje de programación derivado de Pascal diseñado específicamente para la creación de aplicaciones de “larga duración”. Desarrollado por iniciativa del Departamento de Defensa (DoD) de los EEUU para desarrollar sistemas empujados y de Tiempo Real. Cualquier desarrollo con el DoD sobre STR tiene que estar especificado en ADA. Actualmente se utiliza la versión ADA95 y el compilador de GNU para ADA es *gnat*.

2.2 LÉXICO

Los identificadores pueden contener letras mayúsculas y minúsculas, dígitos o el guion bajo “_”. Sensor, tiempo_De_activacion. En ADA no se distingue entre mayúsculas y minúsculas.

Las palabras reservadas no pueden ser utilizadas como identificadores (for, while...). Se puede usar el carácter separador con los números para claridad: 123, 150_000 (=150000), 3.141_592_654 (=3.141592654) ...

Se usa el carácter “#”s para indicar un número en otra base. 2#10001100# (En base 2, 10001100 = 140 en Decimal) Los comentarios empiezan por -- y van hasta final de línea.

2.3 TIPIFICACIÓN DE LOS DATOS

- **Tipificación fuerte de datos:** Todos los datos tienen asociado un tipo de dato. Solo se pueden realizar operaciones entre datos del mismo tipo. No se realizan conversiones de tipo, salvo que se realicen de manera explícita. Los resultados dan siempre un resultado del tipo correcto.
- **Tipificación débil de datos:** Los datos pueden no tener tipo de dato asociado. Se realizan conversiones de tipo de manera automática entre tipos de datos “compatibles”. Los resultados pueden tener un tipo de dato no coincidente con las entradas.

Hay problemas con lenguajes con tipificación débil como FORTRAN.

La precisión de los datos numéricos ya que las variables con el mismo tipo pueden tener precisiones diferentes en función de la arquitectura: Enteros: 8 bits, 16 bits, 32 bits, 64 bits. Reales: 16 bits, 32 bits, 64 bits, 80 bits, 128 bits. El rango de trabajo de las variables y la precisión entre números determinan el número de bits.

2.3.1 Tipos de datos (ADA)

- **Números reales: (Coma flotante)** Se especifican el número de dígitos con los que se representará el número real, pero no se indica nada sobre la precisión ni el grado de exactitud sobre el rango.
 - Float: Reales con 6 dígitos de precisión.
 - type Temperatura is digits 5 range 0.0 ... 100.0. ADA reserva los bits necesarios para esos rangos y sino da error.
 - *digits* indica el número mínimo de dígitos para representar el rango.
- **Números reales: (Coma fija)**
 - Ordinarios: Se especifica el grado de exactitud que ha de mantenerse durante todo el rango del tipo.
 - Duration: Se utiliza para control de tiempo. Precisión: 1E-9
 - type Punto_Fijo is delta 0.1 range -1.5 .. 1.0;
 - Decimales: Se especifica el grado de exactitud y el número de dígitos de precisión que ha de tener:
 - type Decimal is delta 0.01 digits 5
 - -999.99 ... +999.99

Definición:

— nombreVariable* : Tipo;

Asignación:

— nombreVariable := valor;

Ejemplo:

```
type Indice is range 1..100;
type Longitud is digits 5 range 0..100.0;
first, last: Indice;
front, side: Longitud;

last := first + 15;           -- correcto
side := 2.0*front;           -- correcto
side := 2*front;             -- INCORRECTO
side := front + 2.0*first;    -- INCORRECTO
side := front + 2.0*Longitud(first); -- correcto
```

2.3.2 Tipos compuestos

- String
- Array
 - type Temperaturas is array (Indice) of Temperatura; Los tipos Temperatura e Indice deben estar previamente definidos
 - type matriz is array (1..10, 1..10) of Float;
- Ejemplos:

```
t: Temperaturas;
t(5) := 1.0;
t := (25.0, 23.0, 23.0, 23.0, 20.0, 23.0, 23.0);
t := (1=>25.0, 5=>20.0, others=>23.0);
```

- Registros: Tipo de dato compuesto que permite agrupar diferentes tipos de datos en un tipo de dato abstracto.

- type TipoRegistro is record
 - variable: Tipo;
- end record;

- Ejemplo:

```
type Operacion is record
  m : Modo;
  temp : temperatura;
end record;

Op: Operacion;
Op.m := Rapido;
Op.temp := 25.0;
Op := (Rapido, 25.0);
Op := (m => Rapido, temp => 25.0);
```

2.3.3 Tipos acceso (Punteros)

Designan valores de otros tipos.

- type operacionAcceso is access Operacion;
- Se pueden acceder a objetos creados dinámicamente:
 - t: operacionAcceso := new Operacion;
- Si no se indica nada, los nuevos objetos se inicializan a null.
- Acceso dinámico:

```
t.m := Lento;
t.all := (m => Lento, temp => 20.0);
t := new Operacion'(m=>Lento, temp => 20.0);
```

2.4 INSTRUCCIONES

- Simples:
 - Asignación: variable := valor;
 - Llamada a procedimiento: funcion(parametros);
- Compuestas:
 - Bloque:

```
• declare
  - A: integer;           -- Parte de declaraciones de variables
• begin
  - Leer(a);
  - Imprimir(a);
• end;
```

- Selección:
 - **if a>b then**
 - max := a;
 - **elsif a<b then**
 - max := b;
 - **else**
 - max := 0;
 - **end if;**

- Selección por casos: Se realiza sobre una variable discreta y hay que cubrir todas las alternativas de la variable.

```

- Case dia is
  - when lunes                => principio_semana;
  - when martes..viernes      => mitad_semana;
  - when sabado|domingo       => fin_semana;
  - when others               => no_hay;
- end case;

```

- Iteración:

```

- Bucle finito:
  - for i in 1..10 loop
    - leer(v(i));
  - end loop;
  - La definición de i es implícita en función del rango.
- Bucle condicional:
  - while a < b loop
    - a:=a+1;
  - end loop;
- Bucle condicional final:
  - loop
    - leer(d);
    - exit when d=0;
  - end loop;

```

2.5 SUBPROGRAMAS

La especificación se declara en una zona declarativa.

- Procedimientos sin parámetros:
 - **procedure** *nombreProc*;
- Procedimientos con parámetros:
 - **procedure** *proc* (*param1: in out Tipo; param2 : in Tipo:= ValorDefecto*);
- Funciones:
 - **function** *nombreFuncion* (*param1: TipoA; param2 : TipoB*) **return** *TipoC*;
- Tres modos de los parámetros:
 - **In**: No se modifica el valor al ejecutar el subprograma. Puede tomar un valor por defecto.
 - **Out**: El valor del parámetro será asignado por el subprograma.
 - **In out**: el valor del parámetro es tanto de entrada como de salida.
- Las funciones solo pueden tomar parámetros de tipo **in**.

El cuerpo del subprograma se coloca tras la zona declarativa.

- Procedimientos:
 - **procedure** *Incremento* (*param1: in out integer; param2 : in integer:= 1*) **is**
 - **begin**
 - *param1 := param1 + param2;*
 - **end** *Incremento*;
- Funciones
 - **function** *maximo* (*a,b: integer*) **return integer is**
 - **begin**
 - **if** *a>b* **then return** *a*;
 - **else return** *b*;
 - **end if**;
 - **end** *maximo*;

2.6 ESTRUCTURA DE PROGRAMAS

Un programa en ADA está formado por unidades de programa. Distintas unidades de programa:

- **Subprogramas:** Funciones y procedimientos.
- **Paquetes:** Unidad lógica para definir una colección de entidades lógicamente relacionadas.
- **Otras:** Tareas, Objetos protegidos, Unidades genéricas.

Las unidades de programa tienen 2 partes: declaración y cuerpo. Un programa en ADA está formado por:

- Un procedimiento principal.
- Otros subprogramas o paquetes escritos por el programador.
- Subprogramas o paquetes precompilados.

3. TRATAMIENTO DE LOS ERRORES DE EJECUCIÓN

Tipos de Errores:

- **En Tiempo de Compilación.** Deben ser detectados por el compilador.
- **En Tiempo de Ejecución (Excepciones).** No los puede detectar el compilador y se producen durante la ejecución de la aplicación (Ej.: División por cero, logaritmo de un número negativo...)

En general, cuando se produce un error de ejecución se genera un mensaje por la pantalla y se aborta el programa. Esto no es aceptable en una aplicación de STR donde no debe afectar que haya o no haya excepción. Los lenguajes con soporte para STR deben proporcionar mecanismos para detectar errores de ejecución y dar una respuesta a los mismos:

- Simple.
- Ejecución rápida (bajo “overhead”).
- Libertad para seleccionar las excepciones ante distintas excepciones. Esto significa elegir qué excepciones controlar y de cuales pasar.

El tratamiento de las excepciones se limita a bloques de código: cada bloque de código deberá hacer algo diferente si se produce una determinada excepción. En ADA:

```

- Declare
  - Subtype Temperatura is Integer range 0..100;
- Begin
  - -- Lectura del sensor de Temperatura y asignación del valor
- Exception
  - -- Manejador
- End;
```

Si un valor se sale “fuera de rango” se lanza la excepción `Constraint_Error`.

En C++:

```
try {  
    // Bloque de sentencias protegidas  
}  
catch (nombre_excepcion) {  
    // Manejador de la excepción  
}
```

No todos los bloques pueden tener manejador de excepciones.

La diferencia principal es que en ADA se usa un solo manejador para todas las excepciones y en C++ es uno para cada tipo de excepción.

3.1 Propagación de la excepción

¿Qué hacer cuando se genera una excepción en un bloque que no posee manejador asociado?

- **1ª Solución:** El compilador avisaría al programador de la ausencia del manejador => Elevada complejidad en tiempo de compilación.
- **2ª Solución:** Buscar los manejadores que estén activos en la cadena de procedimientos en la pila de llamada actual: Propagación de la excepción.

Para limitar la propagación en un punto y evitar que “suba” hasta el procedimiento principal se permite el uso de manejadores “catch-all” que capture todas las excepciones.

3.2 Reanudación vs Terminación

¿Qué hacer con el procedimiento que ha producido la excepción tras la ejecución del manejador?

- **Modelo de Reanudación o de Notificación:** Si el manejador ha podido solucionar el problema, el procedimiento continúa.
- **Modelo de Terminación o de Escape:** El manejador no devuelve el control al procedimiento.
- **Modelo Híbrido:** En cada caso el propio manejador decidirá qué debe hacer (continuar o terminar).

4. ENTORNO DE EJECUCIÓN DE LA APLICACIÓN

4.1 MECANISMO DE CREACIÓN DE UN EJECUTABLE

Habitualmente, la plataforma de desarrollo suele ser diferente de la plataforma de explotación. Para crear las aplicaciones, se utilizan compiladores cruzados.

La mayoría de las veces el ejecutable se introduce en el sistema de explotación de modo off-line: mediante cable, inalámbrico, disquete, ROM, Flash, etc. Las plataformas de explotación suelen estar “vacías” (sin S.O.) por lo que, la aplicación debe incluir todas las funciones del sistema operativo, así como un pequeño kernel que proporcione una funcionalidad mínima del S.O.

En los STR se permite incluir solo aquellas funciones que sean necesarias, el resto no se incluyen => Se reduce el código.

4.2 EJECUTABLE AUTOCONTENIDO (*stand-alone executable*)

Se genera un único fichero ejecutable que contiene todo el código, tanto de la aplicación como de las funciones del S.O. como de las extensiones necesarias.

- **Ventajas:** Rapidez: Se conoce a priori la posición de inicio de todas las funciones.
- **Desventajas:** Aumento del tamaño. Imposibilita la actualización de funciones del S.O. sin parar el sistema.

4.3 EJECUTABLE CON ENLACES DINÁMICOS (*dynamically linked executable*)

Se genera un fichero que contiene solo el código de la aplicación. El resto de las funciones y recursos se encuentran en otros ficheros, que son llamados dinámicamente y llevados a la memoria bajo demanda.

- **Ventaja:** Reducción en el tamaño del fichero de aplicación. Actualización de funciones del S.O.
- **Desventajas:** Velocidad de ejecución. Mayor consumo de memoria. Control dinámico de las dependencias.

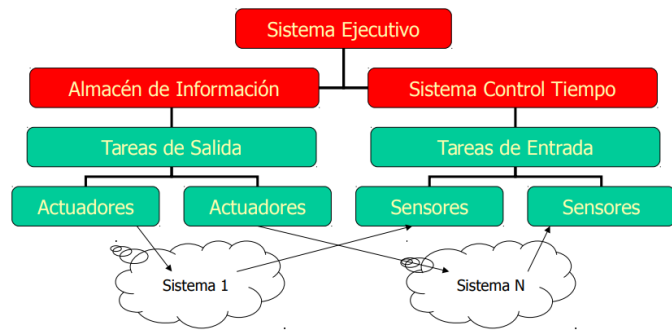
TEMA 3: INTERFACES Y ELEMENTOS HARDWARE

1. ESTRUCTURA DE LOS SISTEMAS DE CONTROL

1.1 SISTEMAS CENTRALIZADOS

Idea original debido a costes de los computadores con un único computador que controla todos los subsistemas de una planta. El tiempo medio entre fallos del hardware en la década de 1960 estaba en el orden de unas pocas horas. Mientras se restauraba el sistema, la planta estaba parada. Por esto el software es MUY restrictivo para evitar fallos debido al software.

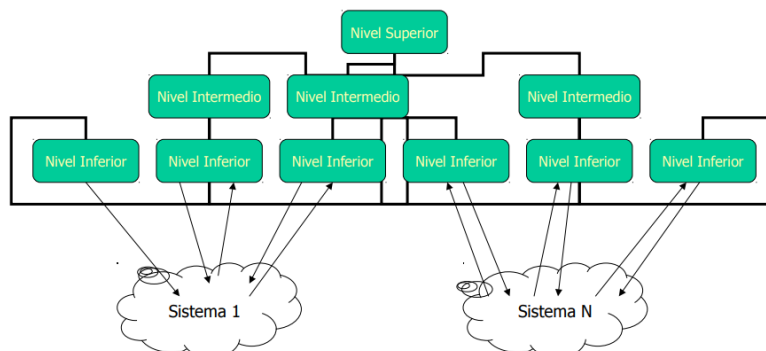
Aumento de la fiabilidad usando esquemas duales. Decaimiento a partir de 1970 con los microprocesadores. Aún en uso de manera más minoritaria y en plantas de muy pocos subsistemas.



1.2 SISTEMAS JERÁRQUICOS

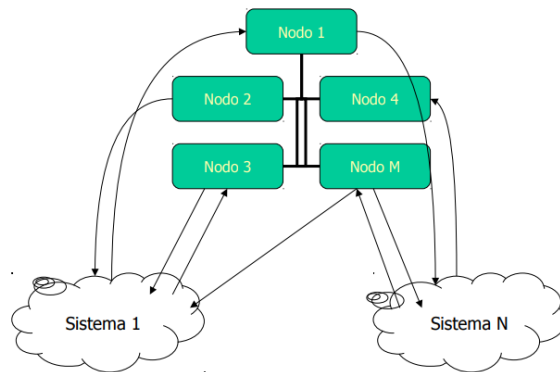
Sigue la estructura natural de toma de decisiones en una compañía. Estructura Piramidal donde un elemento toma las decisiones de alto nivel para problemas muy complejos o que involucren a muchos subsistemas y muchos elementos toman las decisiones rápidas y de bajo nivel para problemas simples.

Es una primera aproximación a la distribución de cargas. Puede presentar determinados problemas de bloqueo en caso de fallos en los elementos de control de nivel superior.



1.3 SISTEMAS DISTRIBUIDOS

Son muchos pequeños elementos de cómputo interconectados, con capacidades similares y responsabilidades de toma de decisión similares. El trabajo se divide entre diferentes elementos y se distribuye para su ejecución. Tienen necesidad de comunicación entre nodos. Pueden cambiar dinámicamente de tareas y reestructurarse.



Tienen una alta adaptación ante requisitos variantes y con alta redundancia ante fallos, elevada dificultad y mayor complejidad de hardware y de software. Es habitual mezclar jerárquico y distribuido.

2. MICROPROCESADORES, MICROCONTROLADORES Y PROCESADORES ESPECIALIZADOS

2.1 MICROPROCESADORES

Se utilizan en sistemas de propósito general. El rendimiento viene determinado por la potencia de proceso, el ancho de banda E/S, la estructura de interrupciones y la capacidad de Almacenamiento Masivo.

Las ventajas son la alta potencia de cálculo y la versatilidad. Los inconvenientes son el alto coste, que necesita más hardware para funcionar y las altas necesidades de consumo energético.

2.2 MICROCONTROLADORES

Procesadores que tienen integrados dentro del mismo chip todos los elementos que componen un computador: CPU, memoria e interfaces E/S. Incluyen elementos específicos para el control de sistemas como los conversores A/D, D/A, temporizadores, puertos serie/paralelo.

Las principales diferencias arquitecturales son las operaciones de manipulación de bit, rápido reconocimiento y procesamiento de interrupciones como direcciones de salto fijas, operaciones de memoria simples rápidas: RAM interna y alta velocidad de cambio de contexto como los bancos de registros.

Las ventajas son la integración completa de un sistema con poco hardware adicional, coste reducido y bajo consumo energético. Los inconvenientes son la capacidad computacional reducida, menor número de herramientas software gratuitas disponibles y necesidad de un sistema “anfitrión” para desarrollar el sistema externamente.

2.3 PROCESADORES ESPECIALIZADOS

Son procesadores que realizan algunas operaciones o tareas específicas de alta complejidad o para ejecutar algunas tareas críticas. Suelen implementarse mediante arquitecturas RISC para facilitar la verificación formal. Para aumentar el rendimiento hay que abandonar la filosofía de arquitectura Von Neumann con procesadores digitales de señales o procesadores específicos: Codiseño Hardware/Software, SoC (System-on-Chip), etc.

Las ventajas son que permiten ajustarse a requisitos temporales muy exigentes. Las desventajas son que son muy caros, pocas herramientas software de soporte y hay que construir todo el software de sistema: drivers, etc.

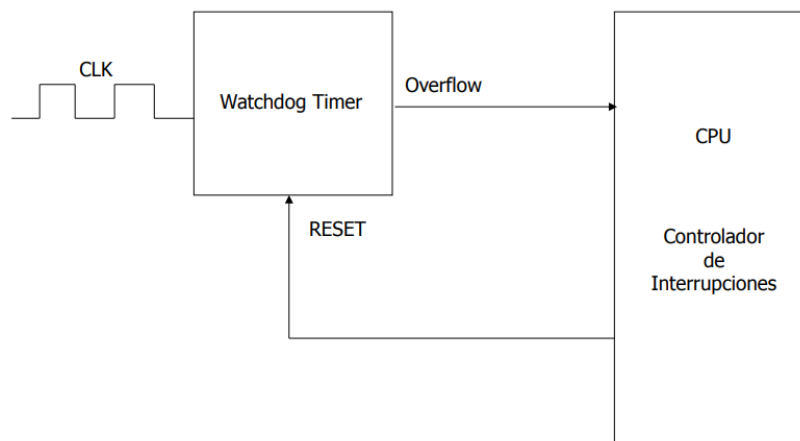
3. TEMPORIZADORES

3.1 RELOJ DE TIEMPO REAL

Puede ser tan complejo como un reloj con múltiples alarmas programables o tan simple como un generador de pulsos. Suelen tener alta precisión. Sirven para medir el tiempo transcurrido (en ticks de reloj) con muy bajo error de estimación. Suelen tener asociada una señal de interrupción para que el sistema ejecutivo pueda tener un control exacto del tiempo transcurrido (en segundos).

3.2 WATCHDOG TIMER

Consiste en un contador que se va incrementando con una cierta frecuencia, pudiendo ser reinicializado y puesto a cero. El desbordamiento en la cuenta del contador se produce en aquellos casos en los que se sobrepase un cierto límite temporal antes de que se haya reinicializado el contador. La señal de desbordamiento suele estar enlazada con la señal RESET del procesador o simplemente conectada a una interrupción que es procesada. Este mecanismo tan simple sirve para detectar lazos muertos y restaurar el sistema a un estado válido, permitiendo a la CPU continuar el procesamiento.



4. TÉCNICAS DE TRANSFERENCIA DE DATOS

Prácticamente todas las tareas de un STR involucran transferencias de datos entre la CPU y dispositivos (sensores o actuadores) externos. Estas transferencias deben ser sincrónicas con la CPU Y CON Diferente velocidad CPU/dispositivos externos. Tienen poca eficacia en casos de control directo de los dispositivos externos directamente por la CPU: muchos ciclos de espera hasta que los dispositivos tengan los datos disponibles. Diversas técnicas de manejo de transferencias como muestreo, interrupciones y DMA.

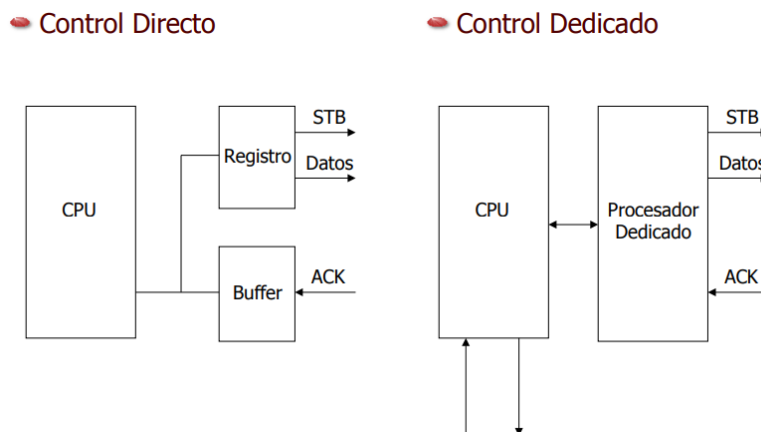
Ejemplo Acceso a una Impresora

Interfaz simple de una impresora (2 señales de control y 1 de datos) 2 señales de control:

- STB: Indicación a la impresora de que tiene datos para impresora.
- ACK: Indicación a la CPU de que ya se ha leído el dato y se puede pasar a procesar el siguiente.

Proceso escritura:

- Escribir el dato en el registro. Activar STB.
- Esperar a ACK para poner un nuevo dato.



4.1 MUESTREO (POLLING)

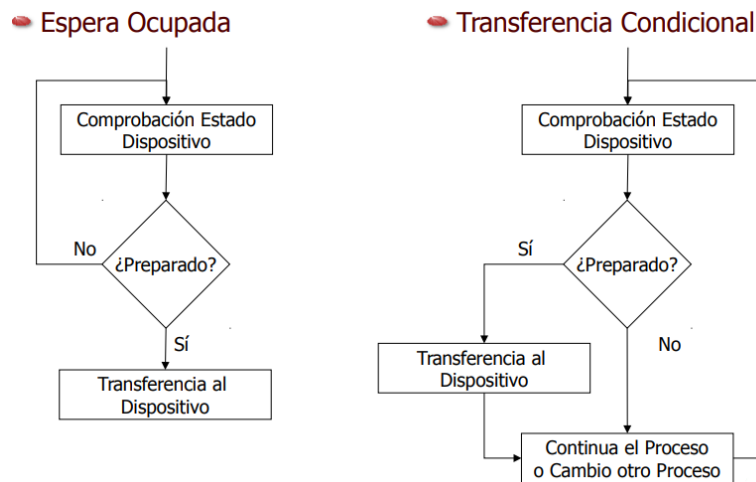
La CPU se encarga de ir chequeando un valor mediante un bucle que es modificado externamente. IN es leer y JZ es salto si no es igual.

```
POLLING: IN AL, DIR_ACK      ; Espera ocupada
          AND AL, 01
          JZ POLLING

          MOV AL, DL          ; Envio de dato
          OUT DIR_DATOS, AL

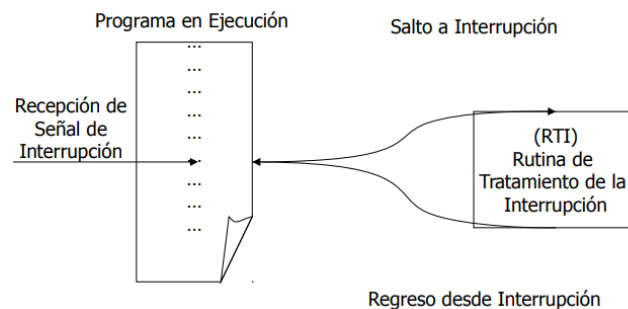
          MOV AL, 01          ; Indico que hay un dato nuevo
          OUT DIR_STB, AL
```


Se pueden dar dos mecanismos principales: Espera ocupada (busy wait) Transferencia condicional (no bloqueante).



4.2 INTERRUPCIONES

Mecanismo que permite interrumpir temporalmente el flujo de ejecución de un programa para ejecutar un trozo de otro programa y continuar la ejecución de programa original de manera transparente a éste.



Permiten sincronizar dispositivos lentos, delegando en un procesador específico la gestión mientras que la CPU continua la ejecución de otras tareas. Controlan otras señales hardware:

- Reloj de Tiempo Real.
- Alarmas de sensores.
- Mantenimiento del sistema.
- Fallo en el Hardware.
- Fallo en la Alimentación.
- Soporte de Depuración.
- Entrada al Sistema Operativo.

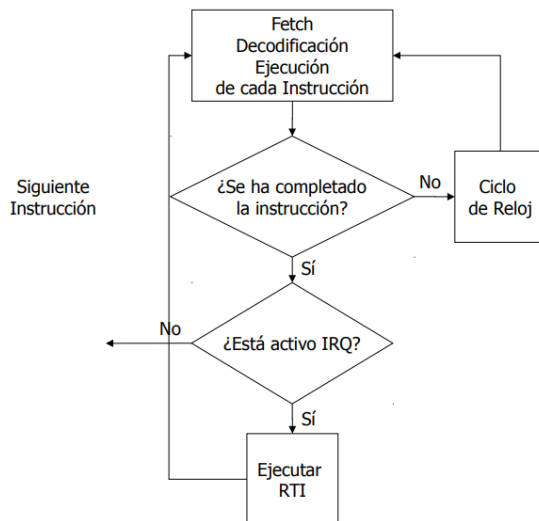
Pueden controlar software, pero lo normal es hardware ya que suelen ser interrupciones eléctricas como un láser en una hab. Cuando se produce una interrupción, la gestión de ésta es transparente al programa en ejecución. Para ello se almacenan los registros de la CPU y al terminar de ejecutarse la RTI se restauran los registros de la CPU con el valor almacenado.

Hay 3 posibilidades de qué almacenar:

- **Todos los registros:** Ineficiente salvo que sean pocos registros.
- **Contador de Programa y Flags:** Almacenar el resto de los registros utilizados es tarea del programador.
- **Ningún registro:** Solo es útil si no se va a volver (Ej. Fallo crítico de alimentación).

El programador de la interrupción es el encargado de restaurar los registros almacenados antes de retornar de la RTI. (Restauración en orden inverso al almacenamiento. Pila).

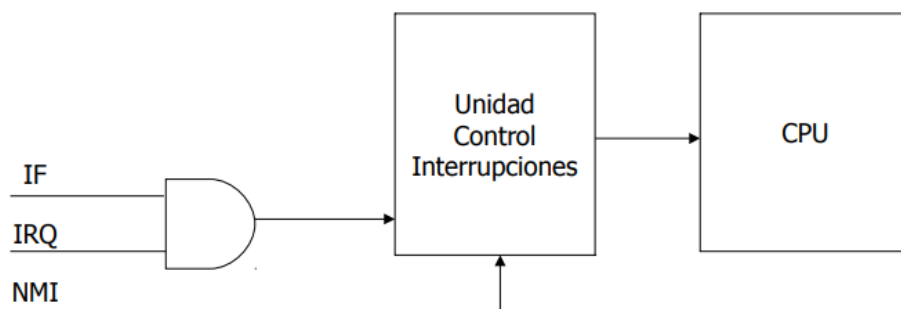
Si se desea imposibilitar otras interrupciones simultáneas, desactivar las interrupciones (CLI). Antes de salir, volver a habilitar las interrupciones (STI) y retornar mediante el retorno especial de interrupciones (IRET) en lugar de a RET porque tiene 2 valores en lugar de uno.



Se puede almacenar en 3 lugares dependiendo de la familia del microprocesador y microcontrolador (NO es elegible):

- Grabar contenido en zona especial de memoria.
- Grabar contenido en la pila.
- Tener varios bancos de registros auxiliares idénticos. Esta es la más eficiente porque se realiza a nivel de HW y no hay que grabar nada en otro sitio.

Las CPU suelen tener 2 señales de interrupción: IRQ (Interruption ReQuest) que se puede habilitar/deshabilitar y NMI (Non Maskable Interruption) que no se puede deshabilitar. IRQ se puede habilitar/deshabilitar por software.



Cuando CLI para, se pone IF a 0 y se bloquea la interrupción.

Para determinar que RTI:

- Transferir el control a una dirección fija.
- Cargar el Contador de Programa (IP) con una dirección contenida en un registro o palabra de memoria.
- Generar una señal de salida para capturar parte de la dirección de memoria donde se encuentra la dirección de la RTI.
- Generar una señal de salida para capturar una instrucción (son solicitudes) dada por un dispositivo externo.

En las 2 primeras, el Hardware solo informa que se ha producido una interrupción, debe determinarse cuál vía software. En las 2 últimas, el Hardware externo identificará la interrupción y saltará directamente a la RTI asociada.

Las interrupciones no determinadas por el hardware, se determinará mediante el software cual ha sido la causa de la interrupción. Se chequeará mediante polling eficiente porque se sabe que, al menos, hay 1 señal activa.

Si hay más de una señal activa (más de una interrupción), ya no es tan eficiente y tenemos mecanismos reducción tiempo:

- Chequeo de las interrupciones más habituales primero.
- Chequear interrupciones que requieran una respuesta rápida.
- Si todas las interrupciones tienen el mismo tiempo de respuesta, rotar el orden de chequeo para garantizar que se atienden todas uniformemente.

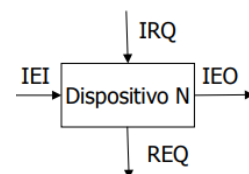
Usando polling el método de prioridad puede ser tan flexible como se desee.

En las interrupciones vectorizadas por el hardware, el hardware es el encargado de informar qué causa produjo la interrupción. Las interrupciones están asociadas a un dispositivo externo y tienen asignados mecanismos de acceso a las RTI asociadas. Necesidad de un mecanismo de arbitrio y asignación de prioridades entre distintas fuentes de interrupción como son el “Daisy Chain” y el codificador de prioridad.

4.2.1 Daisy Chain

Existen varias modalidades de 2, 3 y 4 hilos. Cada dispositivo en el sistema tiene asociado una unidad que posee 4 pines:

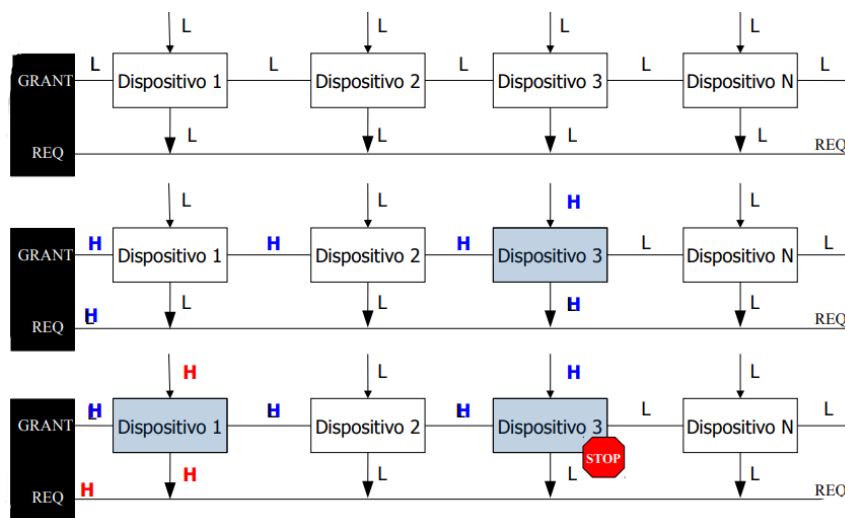
- 1 pin de solicitud de interrupción (IRQ) [Entrada Externa].
- 1 pin de solicitud de petición de ejecución RTI (REQ).
- 2 pines para la estructura “Daisy Chain”:
 - IEI (Interrupt Enable In)
 - IEO (Interrupt Enable Out)



IEO estará activo si IEI está activo o si dicha unidad no tiene IRQ activo. Si un dispositivo está solicitando interrupción (IRQ), si su IEI está activo, entonces IEO pasa a inactivo hasta que finalice la ejecución de la RTI para impedir la ejecución de Interrupciones de menor prioridad. Si su IEI está inactivo, esperar hasta que se active.

El funcionamiento es el siguiente:

- Cuando un dispositivo recibe una solicitud de interrupción, IRQ se activa.
- El dispositivo solicita el control activando el buscador común REQ.
- El árbitro recibe la solicitud y activa la salida GRANT (IEI del primer dispositivo):
 - Si GRANT ya estaba activa, la desactiva una iteración y la activa después.
 - Si GRANT no está activa, la activa.
- Si el primer dispositivo no está solicitando la interrupción, activará IEO hasta que el dispositivo que lo solicite la reciba en cadena.
- Cuando se le concede al dispositivo solicitante, desactiva el REQ. El árbitro no desactiva la salida GRANT de activación.
- Cuando el dispositivo finaliza su rutina, desactiva IRQ y el árbitro desactiva GRANT.



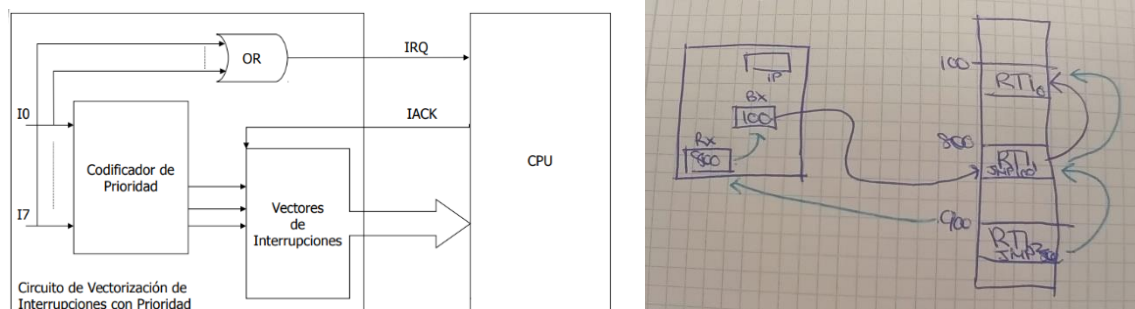
La prioridad viene determinada por la posición en la lista. La prioridad está fijada previamente y no puede ser modificada vía software. Hay que tener mucho cuidado con los tiempos de propagación y con la inhibición de interrupciones para poder garantizar que se garantizan las prioridades.

4.2.2 Codificador de Prioridad

Se solicita interrupción al sistema ante cualquiera de las N líneas que se activen y se genera un código de M bits especificando el número de la línea de interrupción activa.

En el caso de que haya más de una línea activa el codificador con prioridad generará el código de una de las líneas de acuerdo con la prioridad preestablecida.

Cuando se produce el reconocimiento de la interrupción se leerá por el bus de datos un valor que será distinto dependiendo de la interrupción más prioritaria que esté pedida.



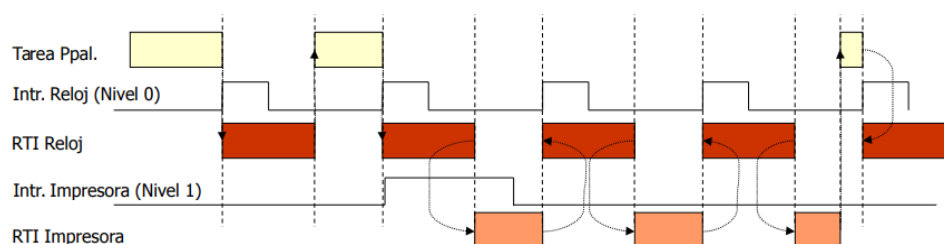
4.2.3 Vectores de respuesta a las interrupciones

El vector generado como respuesta a la interrupción puede ser:

- Una instrucción (habitualmente JMP/CALL o IRET).
- La dirección de la RTI.
- La dirección donde está almacenado el puntero a la RTI.
- Parte de la dirección de la RTI.
- Parte de la dirección donde está almacenado el puntero a la RTI.

4.2.4 Interrupciones Multinivel

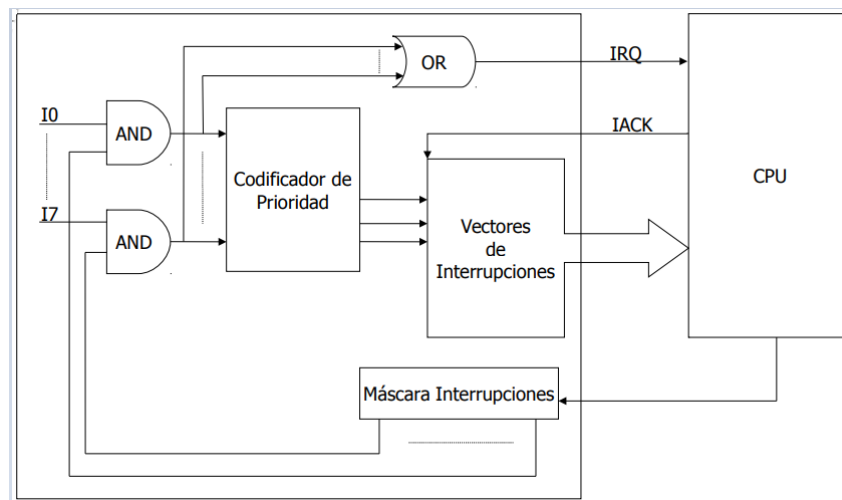
En los STR no es admisible las interrupciones en un único nivel de prioridad. Se debe poder interrumpir el procesamiento de una interrupción por otra interrupción de más alta prioridad. Sin embargo, se debe impedir que una interrupción de menor nivel pueda interrumpir a otra con mayor prioridad (enmascaramiento).



4.2.5 Enmascaramiento de Interrupciones

Utilizando la estructura “Daisy Chain” el enmascaramiento de las interrupciones es automático y se produce al bloquear un dispositivo de alta prioridad todos los de baja prioridad (desactivando su IEO).

Utilizando la estructura codificación de prioridades es mucho más versátil, ya que utilizando un registro de máscara podemos inhibir cualquier interrupción (no solo las prioritarias, sino cualquiera). Permite desactivar la interrupción asociada a una alarma: una vez se reconozca, ya no es necesario volver a tratarla y así no se consume tiempo de procesamiento.



4.2.6 Rutina de Tratamiento de la Interrupción

- Deshabilitar interrupciones.
- Guardar el entorno de trabajo y registro de máscaras.
- Colocar el nuevo registro de enmascaramiento.
- Habilitar interrupciones.
- Ejecutar la RTI en sí.
- Deshabilitar las interrupciones.
- Colocar el registro de enmascaramiento antiguo.
- Colocar los registros almacenados previamente.
- Habilitar de nuevo las interrupciones.
- Retornar de la RTI.

```
RTI_INTx proc far
    CLI                ; Inhabilitamos Interrupciones: No se admiten más
                      ; interrupciones. (Opcional)

    PUSH EAX           ; Guardar los registros que se utilicen dentro
    PUSH EBX           ; de la RTI. (Opcional)
    ...

    ; Código de la RTI
    ...

    POP EBX            ; Antes de salir, restauramos el contenido de los
    POP EAX            ; registros en orden inverso al almacenado.

    STI                ; Habilitamos Interrupciones: Volvemos a admitir
                      ; interrupciones.

    IRET               ; Retorno de Interrupción:
                      ; Saca 3 de la pila Flags, IP, CS

RTI_INTx endpN
```

4.3 ACCESO DIRECTO A MEMORIA (DMA)

Consiste en transferir datos desde/a el dispositivo de E/S directamente a la memoria principal sin pasar por la CPU. También permite mover/copiar bloques de memoria de una zona a otra de la memoria.

El controlador de DMA es el encargado de estas transferencias. Es un procesador dedicado y eficiente, ya que evita pasar por la CPU y permite que ésta se dedique a otras tareas. Para que el DMA pueda iniciar las transferencias, la CPU debe ceder el “mastering” de los buses:

- CPU cede el control de los buses hasta que el DMA haya finalizado todas las transferencias y puede provocar problemas de T° Real.
- CPU cede el control de los buses y puede recuperarlos solicitándolos al DMA antes de que éste finalice completamente.

Hay tres métodos principales de transferencia de datos:

- **Modo ráfaga (“burst mode”)**: El controlador DMA solicita los buses a la CPU y realiza la transferencia completa. Puede afectar al tiempo de respuesta del sistema en el caso que se produzca un evento que deba de ser atendida inmediatamente.
- **Modo distribuido**: El controlador DMA solicita los buses a la CPU para hacer una única transferencia (de uno o varios bytes) y cede el control rápidamente a la CPU. Solo afectaría a STR que utilice “busy waiting” para el cálculo de tiempo.
- **Modo oculto (“cycle-stealing”)**: El controlador DMA toma el control de los buses cuando la CPU no está siendo utilizada. Es la que menos afecta al tiempo del sistema, pero es la más lenta y delicada. Necesita Hardware especial que detecte esos ciclos “ociosos”.

4.4 COMPARACIÓN ENTRE LAS DISTINTAS TÉCNICAS

El “polling” es el método más sencillo para transferir datos en términos de facilidad de programación y chequeo del programa.

El mecanismo de interrupciones es más eficiente que el “polling”. Pero los programas son menos estructurados. Son difíciles de chequear (los errores dependen del tiempo). Las interrupciones son el método más eficaz para dispositivos que no transfieren grandes bloques de datos de una vez.

Si se necesitan altas velocidades de transferencia, las interrupciones no son eficientes: se producen muchos tiempos muertos entre que se produce la interrupción hasta que se ejecuta la rutina de interrupción. En este caso, lo más eficiente serían las transferencias DMA.

TEMA 4: CONCURRENCIA Y SINCRONIZACIÓN

1. PROGRAMACIÓN CON MEMORIA COMPARTIDA

1.1 CONCEPTOS BÁSICOS DE PROGRAMAS CONCURRENTES

Dos o más procesos que se ejecutan simultáneamente trabajando de manera conjunta para realizar una tarea. Las tareas son independientes, cooperativas y competitivas. Su comportamiento puede variar a lo largo del tiempo.

Hay una necesidad de que las tareas se ejecuten en paralelo siempre que sea posible y de manera ordenada cuando necesiten coordinarse. Hay comunicación y sincronización de variables compartidas y paso de mensajes.

1.2 PARADIGMAS DE PROGRAMACIÓN CONCURRENTE

- **Paralelismo iterativo:** Cada for se realiza como una operación concurrente (se acceden a datos independientes).
- **Paralelismo recursivo:** Cada función se ejecuta como una tarea concurrente.
- **Productor/Consumidor:** Existen tareas que producen datos y otras tareas que procesan esos datos de manera concurrente. Comunicación en un solo sentido. Se usa cuando hay dos tareas con distinta velocidad.
- **Cliente/Servidor:** Existen procesos servidores que atienden las peticiones de otros procesos clientes y proporcionan respuestas a los mismos. Comunicación bidireccional.
- **Conjunto de Compañeros (Peers):** Existen procesos independientes que se encargan de realizar una tarea de manera concurrente que se comunican intercambiando datos según los van necesitando.

Dos partes de un algoritmo son independientes si la intersección de sus conjuntos de escritura es el conjunto vacío y si la intersección entre el conjunto de escritura de cada uno de ellos con el conjunto de lectura del otro (en cada caso) es el conjunto vacío.

Se define como conjunto de lectura de un proceso a todas aquellas variables que dicho proceso lee (sin modificar) a lo largo de su ejecución. Se define como conjunto de escritura de un proceso a todas aquellas variables que son creadas o modificadas por dicho proceso a lo largo de su ejecución. Se produce independencia si ambas partes solo acceden a variables locales, si solo “leen” variables compartidas o si “escriben” en diferentes variables compartidas en cada caso.

Cuando dos partes paralelas no son totalmente independientes hay que arbitrar mecanismos para conseguir que la escritura y lectura de las variables compartidas no accedan simultáneamente.

1.3 EJECUCIÓN ATÓMICA DE INSTRUCCIONES

Es una ejecución como una acción única indivisible, es decir, no se puede interrumpir su ejecución en mitad. En ensamblador, las instrucciones pueden durar varios ciclos de reloj, sin embargo, está garantizado que la ejecución de la instrucción completa se realizará sin interrupciones (ejecución atómica).

Al garantizar que una instrucción no es interrumpida hasta que se ejecuta completamente, una instrucción de otro proceso no comenzará a ejecutarse en mitad de la ejecución de otra. Sin embargo, esto no es cierto para instrucciones de alto nivel, ya que pueden estar compuestas por varias instrucciones en ensamblador.

- **Atomicidad o Paralelismo de Grano Fino:** La ejecución atómica de las instrucciones está garantizada mediante mecanismos hardware.
- **Atomicidad o Paralelismo de Grano Grueso:** La ejecución atómica de las instrucciones se garantiza mediante mecanismos software.
 - Paralelismo de Grano Medio: Hace referencia a bloques de código paralelo dentro de una misma tarea. Alto nivel de cooperación/competición, equivale a mucha sincronización.
 - Paralelismo de Grano Grueso: Hace referencia a la ejecución en paralelo de diversas tareas dentro de un mismo sistema. Bajo nivel de cooperación/competición, equivale a poca sincronización.

🔴 Cálculo del máximo en un array: 🔴 Bucle en ensamblador:

```

#define TAM n
int m = 0; // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a) private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            EJECUCION ATOMICA {
                if (a[i] > m)
                    m = a[i];
            }
        }
    }
}

MOV SI,0 ; (i = 0)
BUCLE_FOR: CLI ; BLOQUEO INTERRUPCIONES
MOV AX, A[SI] ; a[i]
CMP AX, [m] ; (a[i] > m)
JBE CONTINUA
MOV [m], AX ; m = a[i]
CONTINUA: STI ; HABILITAR INTERRUP.
INC SI ; i++
; i < TAM => Bucle for
CMP SI, TAM
JB BUCLE_FOR
  
```

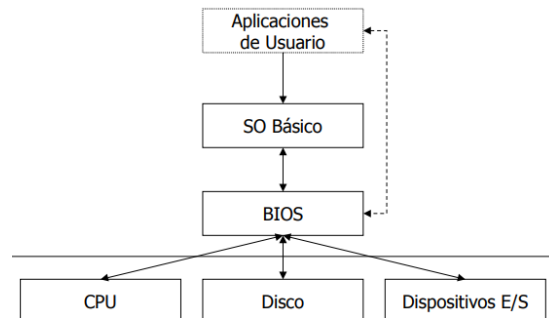
Al ejecutarse de manera atómica todo el cuerpo del bucle es equivalente a ejecutarla secuencialmente. En el secuencial, todos los elementos de la matriz son evaluados desde 0 hasta TAM-1; mientras que en el caso concurrente, el orden de evaluación es arbitrario.

TEMA 5: SISTEMAS OPERATIVOS DE TIEMPO REAL

1. ESTRUCTURA DE LOS SISTEMAS DE CONTROL

1.1 UN SISTEMA OPERATIVO SENCILLO

Los S.O. de control suelen ser sencillos, permitir la interconexión de los programas de usuario con el hardware. Algunos S.O. permiten el acceso directo a rutinas de bajo nivel que acceden al hardware.



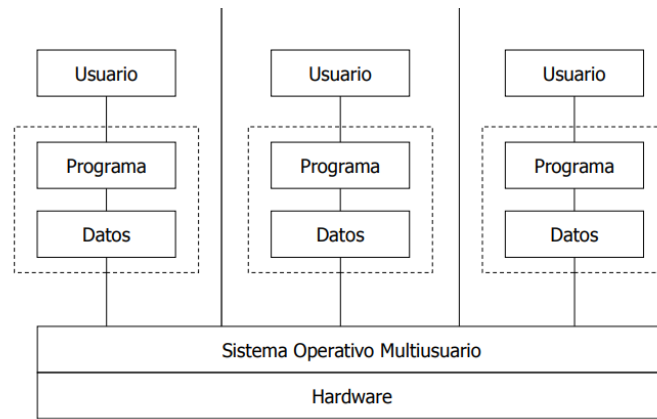
Los componentes de un SO de control sencillo son:

- Las aplicaciones de usuario se comunican con el S.O. Básico (BOS) para interactuar con el hardware.
- El BOS se comunica con el Hardware a través de un conjunto de rutinas implementadas en bajo nivel: BIOS (Basic Input Output System).
- El BIOS contiene los drivers de los dispositivos que manipulan los dispositivos físicos.
- En algunos casos, las aplicaciones de usuario pueden “saltarse” el BOS para solicitar servicios directamente al BIOS.
- La conexión entre lenguajes de alto nivel y el S.O. lo realiza el compilador a través de rutinas run-time.

1.2 SISTEMA OPERATIVO MULTIUSUARIO

Los sistemas multiusuario presentan diferentes usuarios que ejecutan un único programa independiente cada uno de ellos.

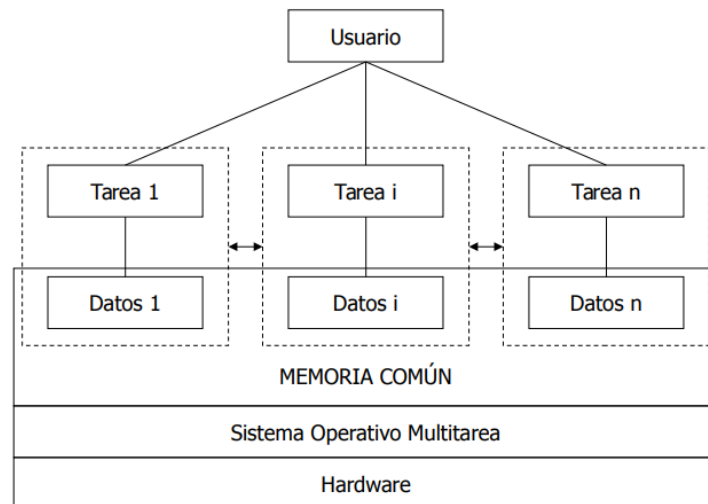
- El S.O. permite a cada usuario ejecutar un programa como si todos los recursos del sistema estuviesen asignados para él.
- La comunicación entre tareas (que serán de diferentes usuarios) se realiza a través del S.O.
- No existen variables compartidas.
- En un instante determinado no se puede saber qué usuario estará haciendo uso de la CPU o qué código estará en memoria, ni cuál entrará después.
- El tiempo de la CPU se repartirá entre cada usuario.
- El programa de cada usuario se ejecuta en un entorno protegido de tal forma que no interfiere uno con otro.



1.3 SISTEMA OPERATIVO MULTITAREA

Hay un único usuario en el sistema que ejecuta diversas tareas que interactúan entre sí.

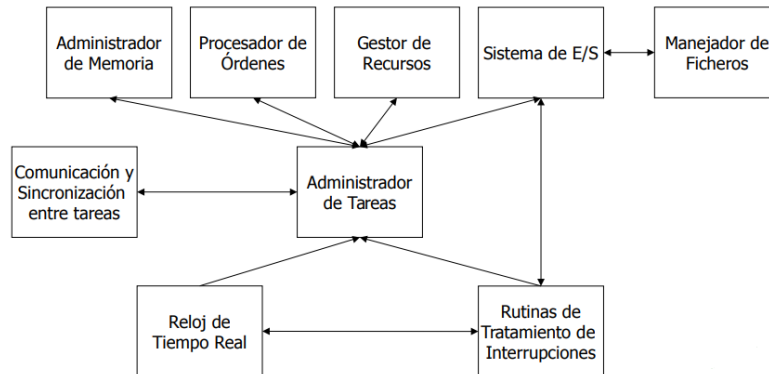
- La interacción entre tareas se realiza mediante datos o variables compartidas.
- Las tareas deberán acceder a las variables garantizando un acceso coherente. El S.O. solo proporcionará mecanismos para poder dar soporte a la exclusión mutua, pero no gestionará el acceso en sí.
- Las tareas no tienen protección de memoria, es decir, que desde una tarea se podría acceder a los datos privados de otra tarea.
- El tiempo de la CPU se repartirá entre cada una de las tareas.



1.4 ESTRUCTURA GENERAL DE UN SO EN TIEMPO REAL

Los S.O. deben asignar los recursos del sistema entre todas las tareas y usuarios, en un SOTR esta asignación es mucho más compleja debido a la presencia de tareas críticas y diferentes niveles de prioridad. Los SOTR deben ser capaces de asignar prioridades a las tareas y planificar el tiempo de CPU utilizando algún esquema de prioridad.

Los SOTR deben permitir la utilización de interrupciones proporcionando un sistema útil y sencillo, a la vez que potente y robusto, para añadir/eliminar rutinas RTI propias. Los SOTR deben proporcionar mecanismos que impidan que las tareas intenten utilizar un recurso al mismo tiempo.



- **Administrador de Tareas:** Asigna la memoria y el tiempo de CPU a las tareas. Este módulo es el núcleo del sistema y también es llamado módulo monitor o executive.
- **Administrador de Memoria:** Realiza el control de la memoria asignada dinámicamente a las tareas según lo solicitan.
- **Comunicación y Sincronización de Tareas:** Permite la comunicación entre las tareas presentes en el sistema, además proporciona los mecanismos necesarios para la sincronización.
- **Gestor de Recursos:** Realiza el control de todos los recursos del sistema excepto el tiempo de CPU y la memoria.

2. ADMINISTRADOR DE TAREAS

2.1 FUNCIONES DEL ADMINISTRADOR DE TAREAS

El S.O. es la tarea de más alta prioridad en cualquier sistema. Toma control cada vez que hay que procesar una interrupción o gestionar una llamada al sistema por parte de las tareas de usuario. El núcleo fundamental se llama administrador de Tareas, módulo monitor o executive. El núcleo tiene las siguientes tareas:

- Almacenar y mantener el estado de cada tarea.
- Planificar el tiempo de CPU para cada tarea según la política de planificación y el tipo de tarea.
- Realizar el cambio de contexto (guardar el estado de la tarea que estaba utilizando la CPU y cargar el estado de la tarea que va a utilizar la CPU, y proceder a la ejecución de la nueva tarea).

Desde el punto de vista del usuario el administrador de tareas es el encargado de:

- Crear las tareas (definir las estructuras en memoria que dan soporte software a las tareas dentro del sistema e ingresarlas dentro de las diversas colas del sistema).
- Destruir las tareas (eliminación del sistema y liberación de la memoria una vez han finalizado su ejecución).
- Soporte de las estrategias de planificación para la asignación de tiempo de CPU a cada tarea.

La creación y destrucción de tareas son llamadas al sistema implementadas en la interfaz del S.O. y del lenguaje de programación.

2.2 ESTRUCTURA INTERNA: PLANIFICADOR Y DESPACHADOR

El administrador de tareas está subdividido en 2 partes:

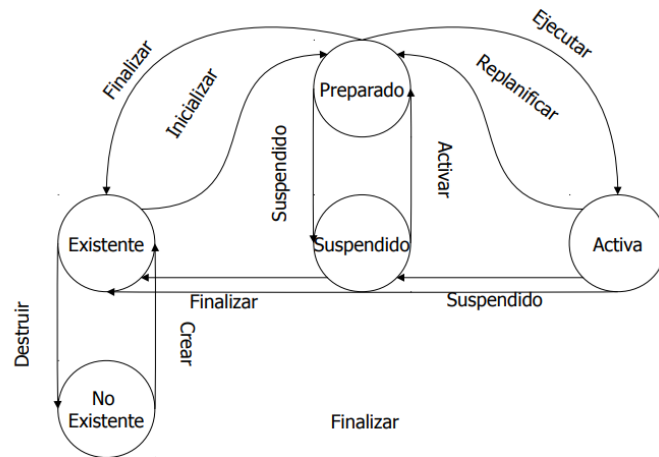
- **Planificador (scheduler):** Determina qué tarea se ejecutará y mantiene el estado de las tareas en el sistema.
- **Despachador o Cargador (dispatcher/loader):** Realiza la conmutación de tareas.

El administrador de tareas se ejecuta cada vez que se produce una interrupción de reloj de sistema. En esa interrupción se determina si hay que continuar ejecutando la tarea actual o si hay que cambiar de tarea. En caso de determinar que hay que cambiar, se llama al planificador y una vez esté determinada la tarea a conmutar, se llama al despachador.

2.3 ESTADOS DE LAS TAREAS

El planificador tiene que controlar el estado de ejecución de las tareas para determinar cuál será la siguiente que se deberá ejecutar o cuál debe ser la primera en desbloquearse. Las tareas se suelen dividir en:

- **Activas o En Ejecución:** Tarea que está ejecutándose. Si es un multiprocesador, habrá más de una.
- **Preparadas o Listas:** Tareas que pueden ejecutarse y están esperando su turno para entrar a ejecutarse.
- **Suspendida o En Espera:** Tareas que están esperando a que algún recurso esté disponible.
- **Existentes:** La tarea existe, pero aún no tiene prioridad asignada.
- **No Existentes o Dormidas:** El sistema ha reservado memoria para la tarea, aunque aún no se ha asignado código de ejecución. Tiene utilidad para Sistemas Operativos con limitación en el número máximo de tareas.

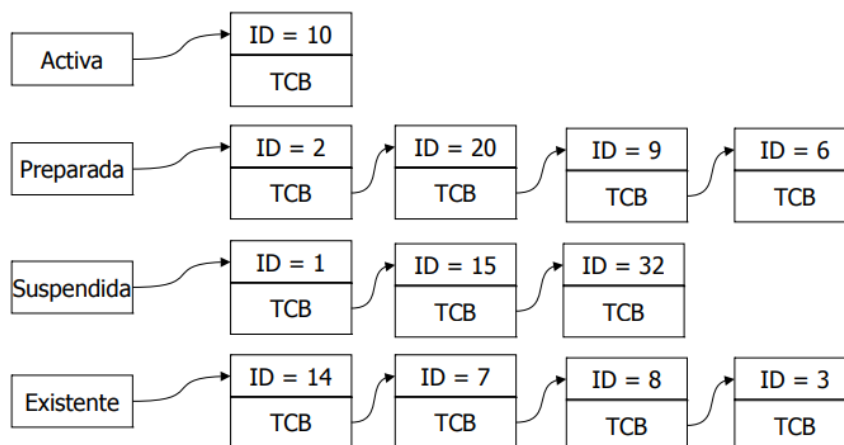


2.4 DESCRIPTORES DE LAS TAREAS

El estado de cada tarea la almacena el S.O. en un bloque de memoria denominado Descriptor de Tareas (TD) o Bloque de Control de Tareas (TCB). Las tareas se almacenan en un conjunto de listas enlazadas. Hay una lista por cada estado posible. Al construirse con listas enlazadas en lugar de con una tabla de descriptores de tamaño fijo, no está restringido a un número máximo de tareas.

El uso de listas enlazadas proporciona rapidez en los intercambios al no necesitar realizar copias, sino cambios en los punteros. En el TCB se almacenan:

- El identificador único de la tarea.
- La prioridad de la tarea.
- El estado de la tarea.
- Una zona de memoria en la que se guarda el contexto de la tarea o un puntero a una zona de memoria externa.
- Un puntero al siguiente TCB de la lista.



3. ADMINISTRADOR DE MEMORIA

3.1 FUNCIONES DEL ADMINISTRADOR DE MEMORIA

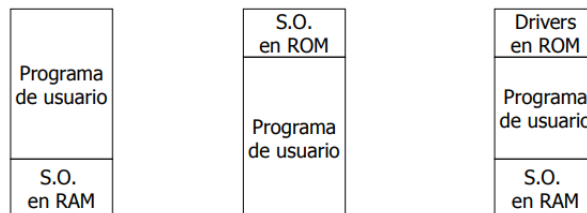
La mayoría de SOTR plantean sistemas de software estático: todas las tareas necesarias se construyen en tiempo de compilación y no aparece o desaparece ninguna en tiempo de ejecución.

En el caso que el SOTR admita la creación de nuevas tareas o su eliminación, o en el caso que el SOTR admita la reserva de memoria dinámica o el realojamiento de las tareas en otras zonas de memoria, es necesaria la presencia de un sistema de administración de la memoria. Este sistema:

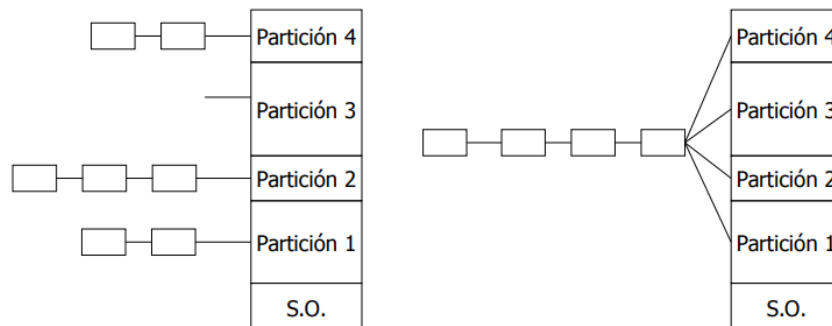
- Gestiona la memoria libre y ocupada del sistema.
- Unifica todos los bloques de memoria libre para hacerlos adyacentes.
- Se encarga de intercambiar tareas o trozos de tareas (datos o código) conforme se van necesitando entre memoria principal y auxiliar.
- Gestiona la memoria dinámica.
- Controla la memoria virtual y el “swapping”

3.2 ADMINISTRACIÓN BÁSICA DE MEMORIA

- **Gestión sin intercambio ni paginación:** Útil en sistemas con un único proceso que consume toda la memoria.



- **Múltiples procesos en particiones fijas:** Útil para sistemas basados en procesamiento por lotes. La memoria se divide en varias particiones de tamaño fijo y diferente entre sí. Puede existir varias colas asociadas cada una a una partición con una lista de posibles tareas que van rotando según se necesita su carga en memoria. Otra opción es una única cola que pueda dirigir una tarea a cualquier partición.



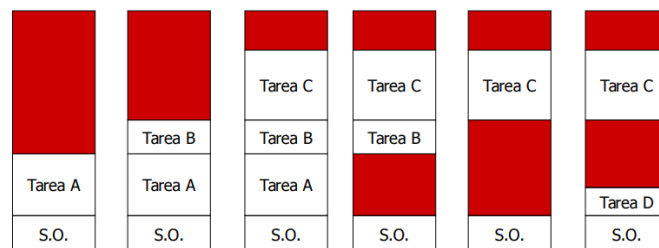
- **Reubicación y protección:** Asociado a la posibilidad de cargar una tarea en diferentes particiones aparece el concepto de reubicación. Las tareas se pueden ejecutar en distintas posiciones de memoria: las referencias de salto y carga han de ser relativas al inicio de la tarea. Para evitar saltos fuera del rango asociado a cada tarea, el sistema hardware debe proporcionar un mecanismo de protección (usualmente basado en códigos de protección que se comprueban vía hardware en cada acceso). La reubicación y la protección se pueden controlar vía hardware con registros de base y de límite.

3.3 INTERCAMBIO DE MEMORIA

En sistemas de tiempo compartido suele haber más usuarios que memoria para todos sus procesos. La solución es mantener en disco los procesos que no caben en memoria y cargarlos conforme se van necesitando para su ejecución. La operación de mover procesos entre el disco y la memoria se denomina intercambio o swapping. El intercambio de memoria se puede realizar con particiones fijas, pero no es habitual y se usa la técnica de particiones variables.

3.4 PARTICIONES VARIABLES

Las particiones fijas malgastan mucha memoria y pueden impedir la ejecución de tareas, aunque el espacio completo libre sea mucho mayor que el que requiere la tarea. Las particiones variables permiten variar dinámicamente el número y el tamaño de las tareas. Mejora la utilización, pero complica la asignación y desocupación de la memoria y el seguimiento del estado de cada zona de memoria. Puede provocar fragmentación de la memoria.

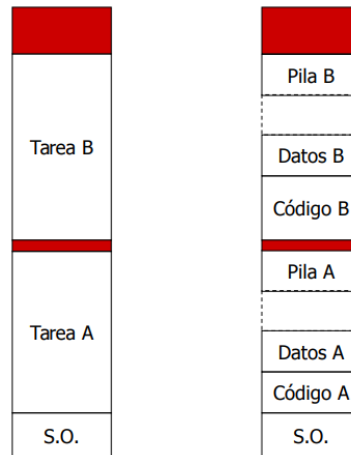


3.5 RESERVA DINÁMICA DE MEMORIA

Las tareas pueden programarse con un número limitado variables de tamaño fijo, pero el uso de variables que puedan variar dinámicamente en tamaño o en número facilita la programación. Si las tareas no van a requerir más memoria que la original, se les puede asignar una partición con un tamaño determinado (que no va a variar a lo largo de la ejecución). Si las tareas van a hacer uso de memoria dinámica hay que concederles más memoria.

- Si hay un hueco junto a la partición, se le asigna a la tarea y continúa.
- Si no hay huecos “ceranos” a la partición, habrá que buscarlos o mover al resto de tareas para compactar la memoria y “abrir hueco”.

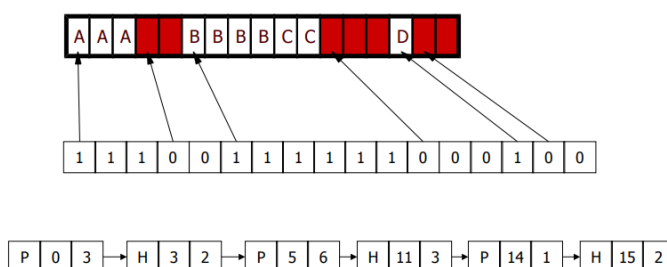
Se suele utilizar la técnica de reserva de bloques de datos contiguos con espacio para crecer (pila hacia abajo, datos/heap hacia arriba).



3.6 GESTIÓN DE LA MEMORIA DINÁMICA

En los SOTR se debe evitar la reserva dinámica siempre que se pueda, porque el beneficio es menor que el coste computacional. Hay que añadir más RAM frente a mayor complejidad en los algoritmos de gestión de la memoria dinámica. Hay tres métodos:

- **Mapa de bits:** Se tiene una tabla donde cada bit representa un bloque de memoria de n bytes. Si es 1 está ocupada, si es 0 está libre. La asignación se consigue buscando una ristra de k bits a 0 seguidos. La liberación se hace poniendo a 0 una serie de bits a 1. La búsqueda de huecos es lenta.
- **Listas enlazadas:** Mantener una lista enlazada con los segmentos ocupados y con los segmentos libres. El segmento contendrá un proceso o el hueco existente entre dos procesos. La asignación será recorrer la lista buscando un segmento libre igual o mayor que la solicitud. La liberación cambiará el valor de ocupado a libre. En el caso de que uno de los dos nodos adyacentes (o los 2) estén libres, se unificarán en uno solo de mayor tamaño. Algoritmos de asignación de memoria:
 - El primer hueco donde quepa (first fit).
 - El siguiente hueco donde quepa (next fit).
 - El hueco donde quepa mejor (best fit).
 - El hueco donde quepa antes (quick fit).



- [illegible]

Las propias tareas pueden controlar qué bloques de programa se encuentran en cada momento en su zona de memoria asignada.

- | | | | |
|-------------------|-----------|-----------|--|
| Tareas de usuario | | | |
| Solapamiento T2 | Overlay 1 | Overlay 2 | |
| Solapamiento T1 | Overlay 1 | Overlay 2 | |
| Raíz T2 | | | |
| Raíz T1 | | | |
| S.O. | | | |

4. PROCESOS E HILOS

4.1 CONCEPTOS BÁSICOS SOBRE PROCESOS

Las tareas son el conjunto de algoritmos que se ejecutan para cumplir alguna misión u objetivo dentro de un sistema informático. Nos referimos a programas a aquellas tareas que no están en ejecución sino almacenados en disco u otro dispositivo de almacenamiento masivo.

Las tareas pueden estar compuestas por uno o varios procesos. Cada proceso está compuesto por código ejecutable, los datos asociados y el contexto de ejecución del proceso (estructuras que sirven para la ejecución del mismo). Los procesos pueden estar compuestos por uno o varios hilos (o hebras) de ejecución.

Un proceso tendrá como mínimo un hilo de ejecución, sin embargo, puede contener varios hilos de ejecución. Todos los hilos dentro de un mismo proceso comparten una gran cantidad de variables y estructuras de funcionamiento.

Todos los hilos de un proceso comparten el mismo espacio de memoria, tanto datos como código. Cada hilo posee un espacio de pila propio. El cambio de contexto entre los hilos de un mismo proceso es mucho más rápido que entre procesos.

4.2 PLANIFICACIÓN DE PROCESOS E HILOS

El S.O. gestiona la lista de tareas para ir proporcionando tiempo de CPU a cada una de ellas para ir progresando en su cometido. La mayoría de S.O. sólo planifican tiempo de CPU para los procesos, ya que el planificador desconoce la existencia de los hilos y presupone que en el sistema solo existen procesos.

El cambio de planificación de los procesos es muy costoso, puesto que hay que almacenar muchos registros y variables del proceso que sale de ejecución, y a su vez cargar todos los registros y variables del proceso que entra en ejecución. Para evitar tener que hacer los cambios de contexto en el S.O., han implementado hilos que realizan tareas del S.O.

Para evitar tener que hacer los cambios de contexto en el S.O., han implementado hilos que realizan tareas del S.O. que se denominan hilos kernel. El S.O. tiene conocimiento de la existencia de estos hilos y es capaz de planificar la ejecución de los diferentes hilos. Los procesos de usuario también pueden utilizar hilos, denominados hilos de usuario. El S.O. no tiene conocimiento de la existencia de estos hilos y por tanto no son planificables directamente. El S.O. planifica tiempo de CPU para el proceso completo y éste tiene que repartir el tiempo de CPU entre los diversos hilos del proceso.

Algunos S.O. proporcionan servicios para crear hilos de usuario que son gestionados y controlados por el propio Sistema Operativo. Estos hilos se denominan procesos ligeros.

Los hilos kernel son objetos ligeros planificables y despachables directamente por el S.O:

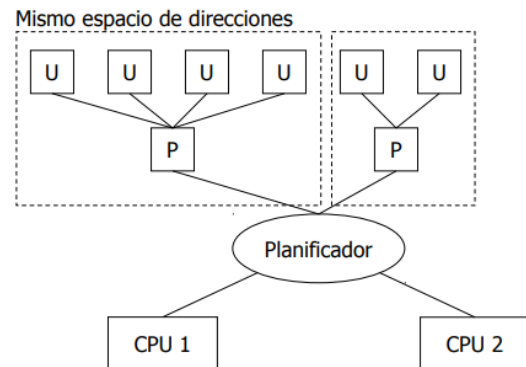
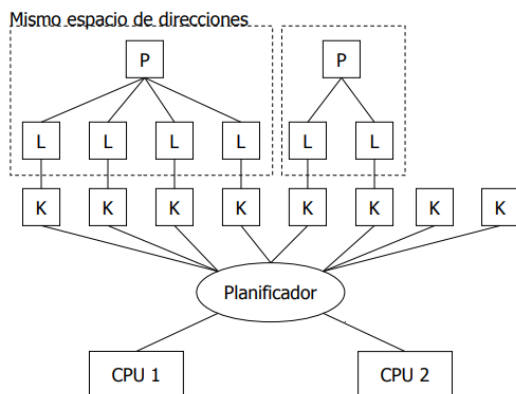
- No consumen demasiados recursos del S.O.
- No necesitan estar asociados a ningún proceso.
- Comparten espacio de direcciones y recursos del S.O.
- Pueden ser creados y destruidos a demanda muy rápido por el S.O.
- Pequeño consumo de recursos: pequeña estructura de datos y una pequeña pila.
- Se suelen utilizar para manejar actividades asíncronas.
- Se suelen utilizar para dar soporte a los procesos ligeros (cada proceso ligero tiene un hilo kernel asociado, aunque pueden existir hilos kernel no asociados a ningún proceso ligero).

Los procesos ligeros proporcionan múltiples hilos de planificación dentro de un mismo proceso:

- Son planificables independientemente y de manera directa por el S.O.
- Cada proceso ligero está asociado a un hilo kernel.
- Elevado consumo de recursos kernel: estructura de datos y memoria para la pila kernel.
- Comparten algunos recursos comunes entre todos los procesos ligeros de un proceso.
- Todos los procesos ligeros comparten el conjunto de manejadores de señales, aunque cada uno tiene su propia máscara de señales.
- No poseen un espacio de nombres global, por lo que son invisibles a otros procesos.
- Son útiles si tienen poco acceso a datos compartidos ya que la sobrecarga por sincronización anula la ventaja del paralelismo.
- La creación, destrucción, bloqueo y desbloqueo de los procesos ligeros son llamadas al sistema muy costosas debido al cambio de modo usuario a modo kernel que se produce en cada llamada.
- Si un proceso tiene muchos procesos ligeros puede llegar a monopolizar la CPU.

Los hilos de usuario:

- No son planificables directamente por el S.O. (éste desconoce su existencia).
- Al no involucrar al S.O. las interacciones entre hilos de usuario son muy rápidas.
- Cada proceso (usualmente mediante una biblioteca de funciones) tiene un pequeño núcleo/planificador que es el encargado de la planificación de los hilos de usuario.
- Cada hilo de usuario tiene su propia pila y un pequeño contexto. El consumo de recursos es muy pequeño.
- Son muy recomendables si hay mucha interacción entre los hilos de usuario y poco acceso a recursos del sistema, puesto que podrían bloquear a todos los restantes hilos.



4.3 GESTIÓN DE BLOQUEOS EN PROCESOS E HILOS

El bloqueo de un proceso provoca la retirada de dicho proceso de la lista de “preparados” y se pasa a la lista de “bloqueados”.

En los casos de bloqueo de hilos kernel o de procesos ligeros, el S.O. conoce la existencia de dichos hilos y es capaz de planificar otro hilo diferente que no esté bloqueado, pasando el hilo bloqueado a la lista de “bloqueados”.

En el caso de hilos de usuario, el S.O. no conoce la existencia de dichos hilos y, por lo tanto, no puede planificar otro hilo del mismo proceso, sino que bloquea el proceso completo.

En el caso de hilos de usuario, si un hilo realiza una llamada al sistema bloqueante (por solicitud de un recurso no libre), el S.O. bloqueará el proceso padre, impidiendo la ejecución de ningún otro hilo.

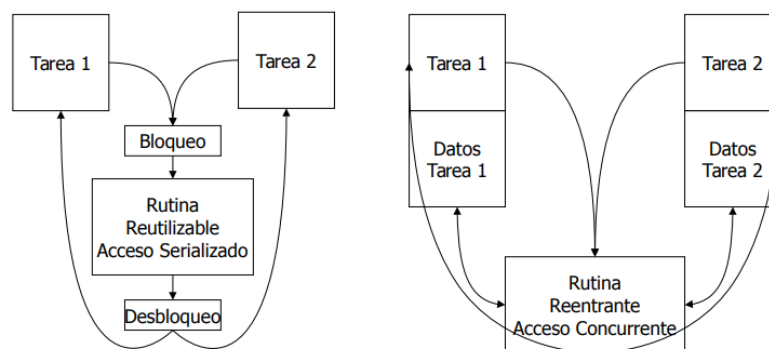
Si la biblioteca de funciones proporciona mecanismos de comprobación de recurso, sin realizar la solicitud efectiva, podrá impedir el bloqueo de todo el proceso, realizando sucesivas solicitudes al S.O. del recurso hasta que esté libre (mediante llamadas no bloqueantes), planificando internamente otro hilo que no esté bloqueado.

En el caso de hilos de usuario, los mecanismos de sincronización no podrán ser del S.O. sino de la propia librería, para controlar qué hilos están en ejecución, preparados o bloqueados.

4.4 REENTRANCIA EN PROCESOS E HILOS

Se dice que un código es reentrante si admite dos o más llamadas concurrentes al mismo código. Todos los códigos que puedan ser llamados por diferentes procesos deben ser reentrantes, todas las RTI deberían ser reentrantes. Para implementaciones del modelo cliente-servidor (o también en otros paradigmas) distintos procesos o hilos pueden realizar llamadas a un mismo código. Este código debe garantizar que las respuestas son correctas en todo caso:

- **Código Reutilizable con Acceso Serializado (Exclusión Mutua):** Existe un único código con datos asociados y que su uso está restringido a una tarea cada vez. Estas tareas no pueden ser interrumpidas hasta que finalicen su funcionalidad, tras lo que pueden pasar a servir a otra tarea diferente. Esto se logra con exclusión mutua.
- **Código Reentrante puro con Acceso concurrente:** Existe un código que no tiene datos asociados o que dentro de su espacio de trabajo no tiene referencias a ninguna tarea en particular. En caso de necesitar almacenar algún dato lo hace en el espacio de memoria de cada tarea llamadora. La reentrancia se consigue mediante el uso de referencias a variables indexadas según la tarea llamadora.



5. COMUNICACIÓN Y SINCRONIZACIÓN ENTRE PROCESOS

5.1 SEÑALES: GENERACIÓN, MANEJO, SINCRONIZACIÓN

El S.O. es el encargado de proporcionar mecanismos a las tareas para generar señales que les permitan comunicarse con otras y con el propio S.O. El S.O. también utilizará las señales para comunicarse con las tareas e informarles de algunos eventos o de excepciones.

El manejo de las señales se realiza a nivel de entidad planificable (procesos, procesos ligeros o hilos kernel). El S.O. es el encargado de enviar las señales a los procesos, indicar que tienen una señal pendiente y de proporcionar los datos para que los procesos puedan obtenerlas.

La sincronización se realiza a nivel de S.O., siendo responsable éste de seleccionar la tarea receptora y desbloquearla (pasarla a la lista de preparados).

5.2 EXCLUSIÓN MUTUA CON ESPERA OCUPADA

Para sincronizaciones de corta duración, la mejor opción es la espera ocupada. En estos casos, el S.O. no saca a la tarea (proceso/hilo) de la cola de "preparados", ya que consumen tiempo de CPU chequeando la condición. Al no estar bloqueado, el S.O. no satura la cola de "bloqueados" y no tiene que buscar la tarea para despertarla.

Este mecanismo no garantiza que, una vez la condición sea cierta, la tarea en espera ocupada pueda continuar inmediatamente, sino que tendrá que esperar a que sea planificado de nuevo y entre en ejecución. No son útiles si la activación de la condición tarda en realizarse mucho tiempo (varios cientos de ciclos o más).

5.3 SINCRONIZACIÓN DE “LARGA DURACIÓN”: DORMIR-DESPERTAR

El S.O. es el encargado de la gestión de dormir/despertar a las tareas. Cuando una tarea solicita “dormir”, el S.O. la pasa a la cola de “bloqueados”. Cuando una tarea desea “despertar” a otra, el S.O. busca dicha tarea en la cola de “bloqueados” y la pasa a la cola de “preparados”.

El S.O. hace uso del sistema de señales entre tareas para despertar la tarea deseada. Los mecanismos dormir/despertar consumen mucho tiempo de CPU debido a la gran cantidad de llamadas al sistema que involucran. No son útiles si se espera que la activación se realice en poco tiempo (unos pocos cientos de ciclos).

5.4 SOPORTE DE SEMÁFOROS Y MONITORES EN SOTR

En los SOTR, el uso de semáforos y monitores está muy penalizado debido al alto consumo computacional que suelen involucrar. Los semáforos (y los monitores) presentan algunos problemas:

- Para garantizar la atomicidad, se aumenta temporalmente el nivel de prioridad al máximo de la tarea llamadora y así no será desplanificada: Problemas con tareas de alta prioridad REAL y necesidad de un S.O. que pueda cambiar dinámicamente las prioridades de las tareas.
- Para evitar la actuación de las interrupciones se inhiben interrupciones temporalmente y problemas con interrupciones y eventos de alta prioridad.
- Los semáforos no están adscritos a ningún proceso y el S.O. tiene que crear hilos kernel o procesos y consumir espacio del S.O. para su manejo.

Los semáforos impiden un control exhaustivo del tiempo de cómputo que puede llevar al incumplimiento de deadlines de tareas de alta prioridad.

5. CONTROL DE RECURSOS

Según el tipo de acceso:

- **Acceso compartido:** El recurso puede ser compartido por varias tareas simultáneamente.
- **Acceso exclusivo:** Una única tarea puede acceder a un recurso a la vez.
- **Acceso mixto:** El recurso puede ser accedido algunas veces de manera compartida y otras veces de manera exclusiva.

La utilización de recursos compartidos: solicitud del recurso, utilización de recurso y liberación del recurso.

Si un recurso está siendo utilizado en modo exclusivo y un segundo proceso intenta acceder a dicho recurso en modo compartido, éste debe esperar. Si un recurso está siendo utilizado en modo compartido y otro proceso intenta acceder a dicho recurso en modo exclusivo, éste deberá esperar a que todos los procesos que están accediendo en modo compartido liberen su utilización.

El S.O. debe tener conocimiento de que hay, al menos, un proceso solicitando acceso en modo exclusivo, para impedir que nuevos procesos accedan al recurso en modo compartido. Como recomendación general, debido a que los procesos pueden bloquearse cuando solicitan un recurso, no deben pedirlo hasta justo cuando lo necesiten y liberarlo tan pronto como sea posible.

5.1 “DEADLOCK”, “LIVELOCKS”, BLOQUEO INDEFINIDO

El sistema puede estar falto de viveza por alguna de estas causas:

- **Deadlock:** Se produce cuando dos procesos no pueden progresar porque necesitan un recurso que posee el otro, la solicitud de los recursos produce el bloqueo de cada proceso, haciendo que no se ejecuten nunca. Ambos procesos estarán esperando bloqueados a que el otro proceso libere el recurso que necesita.
- **Livelock:** Similar a un deadlock, pero las tareas no se bloquean, solo que se mantienen iterando (consumiendo tiempo de CPU) sin que ninguna consiga progresar.
- **Bloqueo indefinido:** Cuando varios procesos intentan acceder a un recurso en exclusiva y nunca lo consiguen. Esto puede ser debido a la política de asignación de recursos (si no impide las solicitudes de recursos en modo compartido una vez hay una solicitud de modo exclusivo).

5.2 CONDICIONES NECESARIAS PARA UN “DEADLOCK”

Para que se produzca un deadlock se tienen que dar 4 condiciones:

- **Exclusión Mutua:** Solo un proceso puede utilizar un recurso a la vez.
- **Mantenimiento y espera:** Un proceso adquiere un recurso mientras que el resto están a la espera de obtenerlo.
- **No apropiación:** Un recurso no puede ser liberado por el S.O. o por otro proceso salvo por el propio proceso que lo ha adquirido.
- **Espera circular:** Existe una encadenamiento de procesos esperando, de tal forma que cada proceso mantenga recursos que están siendo solicitados por el siguiente proceso en la cadena.

Si no se dan las 4 condiciones, no se producirá un deadlock.

5.3 MECANISMOS PARA NO INCURRIR EN UN “DEADLOCK”

Ante el problema de los deadlock hay 3 mecanismos:

- Prevenir los deadlocks.
- Evitar entrar en una situación que pueda llevar a un deadlock.
- Detectar si se ha entrado en un deadlock y recuperarse de él antes de que se produzca el bloqueo efectivo.

Los dos primeros mecanismos son fácilmente implementables: el primero implica al programador, el segundo implica al S.O. El tercer mecanismo es una técnica basada en la teoría de grafos, que trata de comprobar si un grafo es reducible en cada momento y si no lo es, proponer un mecanismo para resolverlo. Es muy complicado y no se implementa en la realidad, solo se utiliza a nivel teórico.

5.4 PREVENCIÓN DE “DEADLOCK”

Para evitar que se produzca un deadlock hay que conseguir que no den alguna de las 4 condiciones:

- **Exclusión mutua:** Para evitar esta condición se puede imponer que el acceso sea compartido o restringir la utilización del recurso en exclusiva a un único proceso.
- **Mantenimiento y espera:** Hacer que los procesos soliciten los recursos antes de comenzar a ejecutarse o en los instantes de ejecución que no tengan recursos pedidos. Esto provoca un uso muy ineficiente de los recursos e incluso al bloqueo en ciertos casos.
- **No apropiación:** Permitir que el S.O. pueda apropiarse de los recursos de un proceso que no responde en un determinado tiempo. Es muy complicado de implementar: requiere guardar el estado antes y después de la apropiación, hecho que a veces es imposible.
- **Espera circular:** Imponer un orden lineal de acceso a los recursos y no admitir solicitudes de un proceso que tenga un recurso adquirido y solicite un recurso que tenga un orden lineal inferior al del recurso ya adquirido.

Evitar la Espera circular mediante acceso lineal.

- Definir el conjunto de recursos del sistema como: $R = \{r_1, r_2, r_3 \dots r_n\}$
- Definir una función de acceso lineal que devuelva el orden de un recurso en la lista de recursos del sistema: $F(r_i) = i$
- Si un proceso tiene un recurso r_j y solicita el recurso r_k se proporcionará acceso a dicho recurso solo si $F(r_j) < F(r_k)$.
- O lo que es equivalente, un proceso que solicite el recurso r_j deberá liberar antes todos los recursos r_i , $i = 1 \dots k-1$; tal que $F(r_i) < F(r_k)$.

5.5 EVITAR “DEADLOCK” USANDO EL ALGORITMO DEL BANQUERO

Diseñado por Dijkstra (1965), se basa en el sistema bancario para la previsión del uso del límite del crédito máximo y sirve para evitar entrar en una situación que pudiera finalizar en un deadlock. Se supone que se tienen varias unidades de un mismo recurso. Un estado es seguro si puede atender al menos 1 solicitud de recursos máximo por parte de un proceso. Un estado es inseguro si no puede atender ninguna solicitud del máximo de recursos por parte de ninguno de los procesos.

Disp.	Utilizados	Necesita
10		
P1	0	6
P2	0	5
P3	0	4
P4	0	7

Disp.	Utilizados	Necesita
2		
P1	1	6
P2	1	5
P3	2	4
P4	4	7

Disp.	Utilizados	Necesita
1		
P1	1	6
P2	2	5
P3	2	4
P4	4	7

Estado seguro

Estado inseguro

5.6 SUBSISTEMA DE ENTRADA/SALIDA

El S.O. debe manejar las peculiaridades de cada dispositivo y presentar a los procesos un mecanismo de acceso unificado a todos los dispositivos. El S.O. también debe sincronizar los accesos concurrentes a los dispositivos.

Los programadores no deberían acceder directamente a los dispositivos, sino a través de llamadas al S.O. para evitar que éste no controle los accesos. Un acceso directo a un dispositivo puede ser más rápido al “saltarnos” el nivel de S.O., sin embargo, podría dejar inconsistente el S.O. o podrían producirse efectos no controlados.

El Subsistema de E/S (en inglés, IOSS: Input Output SubSystem) está dividido en 2 niveles y 3 módulos:

- **Nivel de interfaz con el S.O:** El manejador de E/S.
- **Nivel de enlace con el dispositivo:** El Bloque de control de dispositivo (en inglés, DCB: Device Control Block). Los drivers del dispositivo.

Hay un único manejador de E/S que acepta las llamadas al sistema que realizan las tareas de usuario para solicitar acceso a dispositivos de E/S. El manejador de E/S transfiere la información de la llamada de la tarea de usuario al DCB que es el encargado de procesar dicha llamada y traducirla a funciones específicas del dispositivo. Los drivers del dispositivo contienen las implementaciones de las funciones específicas que permiten acceder al hardware. Existirá un DCB por cada instancia de dispositivo existente en el sistema. Podrá existir drivers específicos por cada dispositivo o incluso se podrá compartir un mismo driver entre varios dispositivos similares, aunque cada uno tendrá un DCB diferente.

La transferencia de datos lo realiza el driver, mediante interrupciones, utilizando la información en el DCB asociado. Los DCB suelen almacenar información de:

- Relativa al dispositivo:
 - Nombre del dispositivo físico y Tipo de dispositivo
 - Dirección del dispositivo
 - Interrupción asociada y dirección de la RTI asociada
 - Estado del Dispositivo
- Relativo a los datos
 - Dirección del búfer de datos
 - Tipo de transferencia (binaria/ASCII)
 - Cantidad de bytes a transferir y Contador actual de transferencia.

